

STRUCT MEMORY-R1: Long-Horizon Conversational Context Access and Updates over Tree-Structured Memory with Reinforcement Learning

Anonymous Author(s)

Affiliation

Address

email

Abstract

Memory-augmented Large Language Models (LLMs) maintain long-horizon memory over conversational history to provide relevant context for future interactions. Conversational information has natural event and topic based structure that a tree can capture better than a flat representation. However, deciding how to traverse relevant tree branches to access memory and where in the hierarchy new facts belong to update memory becomes substantially harder than with flat structure. Reinforcement learning (RL) with Group Relative Policy Optimization (GRPO), which uses group-sampled reward normalization as its training objective, has been shown to improve reasoning ability over complex multi-step trajectories. We propose STRUCT MEMORY-R1, a framework that maintains tree-structured memory from raw conversation and uses RL with GRPO-style question-answering reward to train three components: a *Memory Manager* for tree updates, a *Retrieve Agent* for tree access, and an *Answer Agent* for downstream question answering. On the LoCoMo benchmark, STRUCT MEMORY-R1 achieves the highest overall F1 and LLM-judge accuracy among baseline methods, with particularly large gains on multi-hop (+8.6 F1) and temporal (+14.1 F1) questions over the flat-memory RL baseline.

1 Introduction

Memory-augmented large language models (LLMs) maintain an external memory bank that stores information from past interactions and retrieves relevant context to condition future responses [13]. This paradigm enables long-horizon conversational agents to access information beyond the fixed context window [6]. However, most existing memory systems represent memories as flat, unordered collections and rely on similarity-based retrieval to access them [4, 3, 7].

Flat retrieval treats all memories as independent entries in a global scoring space, ignoring the inherent relationships among them. When a user’s history spans dozens of sessions covering multiple topics, events, and evolving facts, scoring each memory independently leads to *retrieval interference*—semantically similar but contextually irrelevant entries compete with the correct ones. Flat update mechanisms, whether append or overwrite, similarly lack structural awareness: when a new fact arrives, the system cannot determine whether

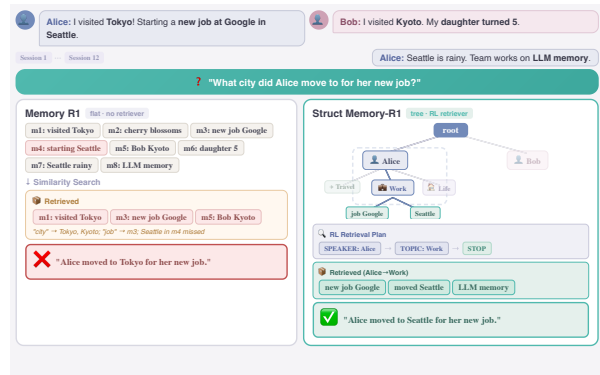


Figure 1: Motivating example. **Memory R1** (left) uses flat similarity search, retrieving “Tokyo” for “city” but missing the job–Seattle link. **Struct Memory-R1** (right) navigates Alice → Work in the tree, correctly finding “Seattle.”

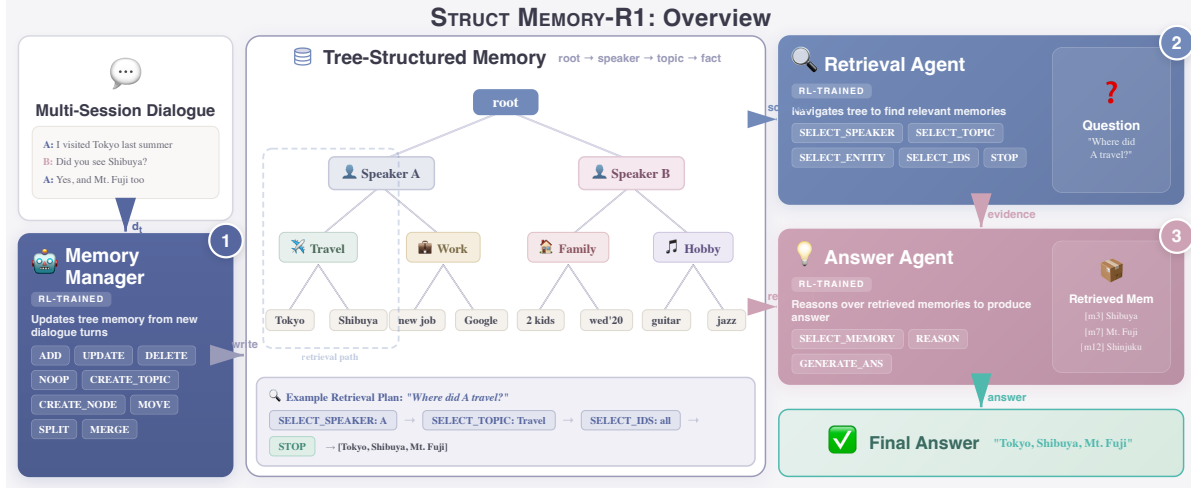


Figure 2: Overview of STRUCT MEMORY-R1. The Memory Manager induces and updates a tree-structured memory from raw conversation. The Retrieve Agent emits a search plan over the tree to select relevant evidence. The Answer Agent produces the final response. All three components are trained with reinforcement learning from downstream question-answering reward.

it extends an existing topic or introduces a new one. As conversations grow, these limitations compound, making consistent memory maintenance increasingly difficult. Conversational information, however, has natural hierarchical structure: discussions revolve around people and topics, topics unfold through events and sessions, and individual facts exist within these groupings. A tree captures this hierarchy, enabling retrieval to first select a relevant branch and then read the leaves within it, and enabling updates to place new facts relative to their structural neighbors rather than into a global pool. Several prior works have explored tree-structured memory representations, including MemTree [8], RAPTOR [9], Lattice [11], and MemWalker [1]. These methods rely on heuristic or LLM-prompted approaches to construct and traverse memory hierarchies.

However, both access and update over a tree involve *reasoning* through possible trajectories: the retriever must decide which branches to explore, and the writer must decide where a new fact belongs in the hierarchy (Figure 1). The quality of these intermediate decisions can only be judged by the correctness of the downstream answer, and direct supervision for such decisions is unavailable in standard QA data. Reinforcement learning from downstream question-answering reward is therefore the natural training signal. This follows the principle established by Search-R1 [2] and Memory-R1 [12]—that information access should be trained from task outcomes—and extends it from flat memory to tree-structured access and updates.

We therefore propose STRUCT MEMORY-R1, a reinforcement-learning framework that learns tree-structured conversational memory from raw interaction history. STRUCT MEMORY-R1 decomposes the problem into three coordinated components (Figure 2):

- A **Memory Manager** that receives a new turn and the current memory, then incrementally constructs and updates the memory tree through structural placement of new facts.
- A **Retrieve Agent** that operates over the induced tree and emits an explicit search plan, selecting branches and leaves rather than relying on global flat retrieval.
- An **Answer Agent** that reads retrieved evidence and produces the final answer.

The decomposition follows the causal chain of the problem: the Memory Manager builds and maintains the tree, the Retrieve Agent navigates it for a given question, and the Answer Agent evaluates whether the retrieved evidence suffices. By separating these roles, STRUCT MEMORY-R1 turns structured memory into a modular reinforcement-learning problem in which tree updates, tree access, and answering are each optimized via Group Relative Policy Optimization (GRPO) against downstream task performance.

In summary, the paper makes three contributions:

- We formalize the memory-augmented LLM setting and argue that conversational memory has hierarchical structure that a tree captures naturally, requiring access and update operations that differ fundamentally from those used on flat memory.
- We propose STRUCT MEMORY-R1, a three-component framework that induces tree structure from raw conversation and trains tree-specific access and update policies with GRPO-based reinforcement learning.
- We evaluate on the LoCoMo benchmark [6], showing that tree-structured memory yields substantial gains on multi-hop (+9.0 F1) and temporal (+13.8 F1) questions while achieving the highest overall F1 among non-oracle methods.

2 Problem Definition

We formalize the setting of a large language model augmented with external memory for long-horizon conversational question answering.

Memory-Augmented Generation. Let π_θ denote an LLM parameterized by θ . At each turn t , the model receives a user query $x_t \in \mathcal{X}$ and has access to an external memory state $M_t \in \mathcal{M}$, where $\mathcal{M}$ is the space of all possible memory configurations. The model produces a response:

$$y_t \sim \pi_\theta(\cdot \mid x_t, R_t), \quad R_t = \text{RETRIEVE}(x_t, M_t), \quad (1)$$

where $R_t \subseteq M_t$ is a subset of memories retrieved as context. The memory is updated after each turn:

$$M_{t+1} = \text{UPDATE}(M_t, x_t, y_t). \quad (2)$$

The objective is to learn the retrieval policy RETRIEVE and the update policy UPDATE such that π_θ maximizes downstream task performance across a sequence of interactions $\mathcal{I} = (x_1, x_2, \dots, x_T)$.

Flat Memory. In prior work [12, 2], the memory state is a flat, unordered collection of N entries:

$$M_t^{\text{flat}} = \{m_1, m_2, \dots, m_N\}, \quad m_i = (\text{key}_i, \text{value}_i), \quad (3)$$

where each entry m_i is an independent fact. Retrieval is performed by scoring each entry against the query independently, typically via embedding similarity or keyword matching:

$$R_t = \text{Top-}k \{m_i \in M_t^{\text{flat}} : \text{score}(x_t, m_i)\}. \quad (4)$$

This formulation treats all memories as exchangeable and cannot exploit structural relationships among them.

Tree-Structured Memory. We propose to organize memory as a rooted tree $\mathcal{T} = (V, E)$ where vertices V are organized into L levels. The root v_0 represents the entire conversation. Internal nodes at level ℓ represent progressively finer groupings (e.g., people $\rightarrow$ topics $\rightarrow$ events), and leaf nodes contain individual facts:

$$M_t^{\text{tree}} = \mathcal{T}_t = (V_t, E_t), \quad V_t = V_t^{(0)} \cup V_t^{(1)} \cup \dots \cup V_t^{(L)}, \quad (5)$$

where $V_t^{(\ell)}$ denotes nodes at level ℓ and E_t encodes parent-child relationships. Each node $v \in V_t$ carries a content string $c(v)$ and a summary $s(v)$ that aggregates information from its descendants.

Under tree-structured memory, retrieval becomes a *search planning* problem. Rather than scoring all entries globally, the retriever selects a path through the hierarchy:

$$R_t = \text{TREERETRIEVE}(x_t, \mathcal{T}_t) = \text{EXECUTE}(\mathcal{T}_t, z_t), \quad z_t \sim \pi_{\theta_R}(\cdot \mid x_t, \text{SCHEMA}(\mathcal{T}_t)), \quad (6)$$

where z_t is a search plan specifying which branches to expand and which leaves to read, and $\text{SCHEMA}(\mathcal{T}_t)$ is a compact representation of the tree structure (node names and summaries, without full leaf content).

Similarly, updates become a *structural placement* problem. Given a new fact extracted from turn x_t , the update policy must decide where in the hierarchy it belongs:

$$\mathcal{T}_{t+1} = \text{TREEUPDATE}(\mathcal{T}_t, x_t) = \text{APPLY}(\mathcal{T}_t, o_t), \quad o_t \sim \pi_{\theta_M}(\cdot \mid x_t, \mathcal{T}_t), \quad (7)$$

where o_t is an update operation that places, merges, or removes facts within the tree.

Learning Objective. Both the search plan z_t and the update operation o_t are intermediate decisions whose quality can only be assessed through the correctness of the final answer y_t to a downstream question. Given a dataset of question-answer pairs $\{(q_j, y_j^*)\}_{j=1}^{N_q}$, we optimize:

$$\max_{\theta_M, \theta_R, \theta_A} \mathbb{E}_{q, y^*} [\mathcal{R}(y, y^*)], \quad y \sim \pi_{\theta_A}(\cdot \mid q, R), \quad R = \text{TREERETRIEVE}(q, \mathcal{T}), \quad (8)$$

where $\mathcal{R}$ is a reward function measuring answer quality and $\mathcal{T}$ is the memory tree constructed by the Memory Manager policy π_{θ_M} . This objective couples memory construction, retrieval, and answering through a shared downstream reward signal.

3 Method

3.1 Overview

STRUCT MEMORY-R1 instantiates the memory-augmented LLM framework defined in Section 2 with three learned components: a Memory Manager π_{θ_M} , a Retrieve Agent π_{θ_R} , and an Answer Agent π_{θ_A} . Each component is parameterized by a language model and trained with Group Relative Policy Optimization (GRPO) [10] using downstream question-answering reward. Training proceeds in stages: the Answer Agent is trained first, then frozen while the Retrieve Agent and Memory Manager are trained in sequence. This staged approach ensures stable reward signals at each phase.

3.2 Memory Bank and Tree Structure

The memory bank $\mathcal{T}_t = (V_t, E_t)$ is a rooted tree that is constructed incrementally from raw conversation. Each node $v \in V_t$ resides at a level $\ell(v) \in \{0, 1, \dots, L\}$ and carries content $c(v)$. The tree is structured such that:

- Level 0: the root, representing the full conversation;
- Levels 1 to $L-1$: internal nodes representing hierarchical groupings (e.g., people, topics, events);
- Level L : leaf nodes, each containing an individual memory entry m_i .

Each internal node v maintains a summary $s(v)$ that aggregates information from its children $\text{Ch}(v) = \{u \in V_t : (v, u) \in E_t\}$. The *schema* of the tree, $\text{SCHEMA}(\mathcal{T}_t)$, consists of node identifiers and summaries at each level without exposing full leaf content, providing a compact representation for the retrieval policy.

The tree is serialized as structured JSON for input to language model policies. This serialization preserves the hierarchical relationships while remaining compatible with standard text-based LLM input formats.

3.3 Memory Manager

The Memory Manager observes the current dialogue turn x_t and the current memory tree $\mathcal{T}_t$, and emits a set of operations to update the tree. We restrict the operation space to four stable CRUD operations:

$$\mathcal{O} = \{\text{ADD}, \text{UPDATE}, \text{DELETE}, \text{NONE}\}. \quad (9)$$

Given turn x_t and tree $\mathcal{T}_t$, the manager samples an operation sequence:

$$o_t \sim \pi_{\theta_M}(\cdot \mid x_t, \mathcal{T}_t), \quad (10)$$

and the updated tree is obtained deterministically:

$$\mathcal{T}_{t+1} = \text{APPLY}(\mathcal{T}_t, o_t). \quad (11)$$

The operation o_t is a JSON object specifying which entries to add, modify, or remove, along with their placement in the tree hierarchy. By restricting to CRUD operations rather than arbitrary structural rewrites, we isolate the benefit of structured memory from the complexity of unrestricted topology editing.

Reward. The Memory Manager is trained with outcome-driven reward. After applying the manager’s operations, the downstream pipeline (Retrieve Agent + frozen Answer Agent) processes the corresponding question. The reward is:

$$r_M = \mathcal{R}(y_t, y_t^*) + \lambda_f \cdot \mathbb{1}[\text{valid JSON}], \quad (12)$$

where $\mathcal{R}(y_t, y_t^*)$ measures answer quality and λ_f is a small bonus for well-formed output. This encourages the manager to write memories that are useful for future question answering rather than those matching local heuristics.

3.4 Retrieve Agent

The Retrieve Agent operates over the memory tree constructed by the Memory Manager. Given a question q and the tree schema $\text{SCHEMA}(\mathcal{T})$, it emits a search plan:

$$z \sim \pi_{\theta_R}(\cdot \mid q, \text{SCHEMA}(\mathcal{T})). \quad (13)$$

The search plan z is a JSON object of the form:

$$z = \{\text{levels} : [\ell_1, \dots, \ell_k], \text{selected_ids} : [id_1, \dots, id_m], \text{stop} : \{0, 1\}\}, \quad (14)$$

where `levels` specifies which branches to expand at each tree level, `selected_ids` names specific leaf nodes to retrieve, and `stop` terminates search. A deterministic executor runs the plan against the tree:

$$R = \text{EXECUTE}(\mathcal{T}, z), \quad (15)$$

returning a set of retrieved memory leaves together with local context (parent summaries, sibling information). This design ensures that retrieval is conditioned on structural locality rather than global similarity scoring.

Reward. The retriever reward combines answer utility with evidence quality:

$$r_R = \mathcal{R}(y_t, y_t^*) + \lambda_{\text{ev}} \cdot \text{HIT}(R, E^*) - \lambda_{\text{cost}} \cdot C(z), \quad (16)$$

where E^* denotes gold evidence nodes (when available), $C(z)$ penalizes search cost, and the primary signal is answer quality under a frozen Answer Agent.

3.5 Answer Agent

The Answer Agent receives the question q and retrieved memories R , and produces a response:

$$y \sim \pi_{\theta_A}(\cdot \mid q, R). \quad (17)$$

This component is trained to condition on retrieved evidence rather than on the full conversation history. During Memory Manager and Retrieve Agent training, the Answer Agent is frozen and serves as the evaluator in the reward loop, ensuring that improvements in upstream components are measured through their effect on answer quality.

3.6 Training with GRPO

All three components are trained using Group Relative Policy Optimization (GRPO) [10], which avoids the need for a separate value network by computing advantages from a group of sampled outputs. For a given input x and policy π_θ , GRPO samples a group of G outputs $\{o_1, \dots, o_G\}$ and computes normalized advantages:

$$\hat{A}_i = \frac{r_i - \mu(\{r_1, \dots, r_G\})}{\sigma(\{r_1, \dots, r_G\})}, \quad (18)$$

where r_i is the reward for output o_i , and μ, σ denote the group mean and standard deviation. The policy is updated by maximizing:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_x \left[\frac{1}{G} \sum_{i=1}^G \min \left(\rho_i \hat{A}_i, \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon) \hat{A}_i \right) - \beta D_{\text{KL}}[\pi_\theta \parallel \pi_{\text{ref}}] \right], \quad (19)$$

where $\rho_i = \pi_\theta(o_i \mid x) / \pi_{\text{old}}(o_i \mid x)$ is the importance ratio, ϵ is the clipping parameter, and β controls the KL penalty relative to a reference policy π_{ref} . This objective encourages the model to produce outputs that yield higher rewards relative to the group average while staying close to the reference distribution.

4 Experimental Setup

4.1 Dataset

We evaluate on LoCoMo [6], a long-horizon conversational memory benchmark containing 10 multi-session dialogues with 1,540 non-adversarial QA pairs across four question categories: single-hop (282), multi-hop (321), temporal (96), and open-domain (841). We exclude the adversarial category following the Memory-R1 evaluation protocol [12]. Following Memory-R1, we use a 1:1:8 train/validation/test split, yielding 152 training, 81 validation, and 1,307 test QA pairs. LoCoMo is well matched to our setting: it requires persistent memory over many turns, supports both single-hop and multi-hop reasoning, and includes temporal and open-domain questions that test different aspects of memory access.

4.2 Model and Hardware

All trainable components use Qwen2.5-3B-Instruct as the base policy model. Experiments are run on a single NVIDIA H200 GPU with 16 CPU cores. This setup supports single-node staged GRPO training and fixed-answer-agent evaluation.

We use GRPO as the default policy-optimization method, following Search-R1 and Memory-R1 [2, 12]. For the Memory Manager and Retrieve Agent, prompts are formatted as one-shot JSON-generation problems with rule-based reward functions. The Answer Agent is trained to produce short answers conditioned on retrieved memories. During Memory Manager and Retrieve Agent training, the Answer Agent is frozen and used only to compute reward.

4.3 Baselines

We organize baselines into three categories.

Context-based. **Full Context** feeds the entire conversation history into the base Qwen2.5-3B-Instruct model without explicit memory or retrieval, relying on the context window alone. **Oracle** uses the gold evidence annotations provided in LoCoMo, representing an upper bound on retrieval quality.

Retrieval-based. **RAG (BM25)** applies BM25 keyword retrieval over a flat memory bank of extracted facts. **RAG (Embedding)** uses text-embedding-3 dense retrieval over the same flat memory bank. **Semantic XPath** [5] generates deterministic XPath-style queries over oracle tree-structured memory.

RL-based. **Memory R1** [12] is a flat-memory system with an RL-trained Memory Manager and Answer Agent, trained on the same base model and the same data split for fair comparison.¹

4.4 Evaluation Metrics

We report three answer-quality metrics. **F1** is the token-level F1 score between the predicted and gold answers, measuring lexical overlap. **B1** is BLEU-1, a unigram precision metric that captures answer conciseness alongside correctness. **J** is the LLM-as-judge accuracy, where GPT-4o mini evaluates whether the predicted answer is semantically correct given the gold answer. All three metrics are reported per question category (multi-hop, single-hop, temporal, open-domain) and as a weighted overall average reflecting the natural category distribution in LoCoMo.

5 Results

5.1 Main Results

Table 1 reports F1, B1, and J across four question categories on the LoCoMo test set.

STRUCT MEMORY-R1 achieves the highest overall F1 (30.4), B1 (25.4), and J (54.4) among non-oracle methods. The gains are concentrated in multi-hop and temporal reasoning, where tree-structured retrieval enables the model to navigate related facts through shared ancestors rather than scoring each memory independently. On multi-hop questions, our method improves F1 by 9.0 points over Memory R1 (31.5 vs. 22.5) and J by 6.1 points (56.2 vs. 50.1). On temporal questions, the improvement is even larger: 13.8 F1 points and 10.6 J points over Memory R1. These categories require combining information across sessions or reasoning about time, both of which benefit from the hierarchical grouping that tree structure provides.

The picture is different for single-hop and open-domain questions. Memory R1 edges out our method on single-hop (27.7 vs. 27.1 F1, 23.6 vs. 21.7 B1, 54.9 vs. 53.7 J), where flat retrieval is sufficient because the answer resides in a single memory entry and tree navigation adds no benefit. On open-domain questions, RAG (BM25) achieves the highest F1 (38.7), B1 (33.9), and J (67.3) by a wide margin. Open-domain questions often require broad factual recall that favors high-coverage keyword matching over focused structural navigation. Our method nonetheless outperforms Memory R1 on open-domain F1 (30.6 vs. 30.0), suggesting that tree-organized memory retains some advantage even for general questions.

¹Memory-R1 does not provide a public codebase. We built a publicly available implementation at https://github.com/YifanLiu2/Struct_Memory_R1.git.

Table 1: Main results on LoCoMo (1:1:8 train/val/test split, 1,307 test QA pairs, adversarial category excluded). F1: token-level F1; B1: BLEU-1; J: LLM-as-judge accuracy (GPT-4o mini). Best non-oracle results per column in **bold**. Oracle results in *italics*.

Method	Overall			Multi-hop			Single-hop			Temporal			Open-domain		
	F1	B1	J	F1	B1	J	F1	B1	J	F1	B1	J	F1	B1	J
Context-based															
No retrieval; answers from raw context or gold evidence															
Full Context	24.2	19.7	45.7	19.7	15.8	40.4	23.9	19.0	45.7	16.2	12.5	37.3	27.0	22.3	48.6
Oracle [†]	<i>31.3</i>	<i>25.7</i>	<i>60.8</i>	<i>34.3</i>	<i>27.9</i>	<i>63.1</i>	<i>31.6</i>	<i>25.9</i>	<i>62.2</i>	<i>35.4</i>	<i>29.7</i>	<i>65.2</i>	<i>29.5</i>	<i>24.3</i>	<i>59.0</i>
Retrieval-based															
Heuristic or similarity-based retrieval; no RL training															
RAG (BM25)	29.9	25.3	49.8	19.0	14.6	28.1	22.6	17.6	30.6	11.2	8.8	25.1	38.7	33.9	67.3
RAG (Embedding)	29.5	24.3	49.6	21.1	15.8	31.5	21.8	17.3	32.4	12.1	8.8	24.2	37.2	31.7	65.1
Semantic XPath	18.5	14.4	39.8	25.9	20.4	53.7	21.3	16.5	45.1	29.8	24.9	49.0	13.4	10.2	31.6
RL-based															
Policy trained with GRPO from downstream QA reward															
Memory R1	27.4	22.7	53.9	22.5	17.4	50.1	27.7	23.6	54.9	20.8	16.5	48.0	30.0	25.2	55.7
STRUCT MEMORY-R1 (Ours)*	30.4	25.4	54.4	31.5	26.8	56.2	27.1	21.7	53.7	34.6	28.6	58.6	30.6	25.8	53.4

[†]Uses gold evidence annotations from LoCoMo (upper bound). *Uses oracle human-judged structured data; results may overestimate performance achievable with automatically induced structure.

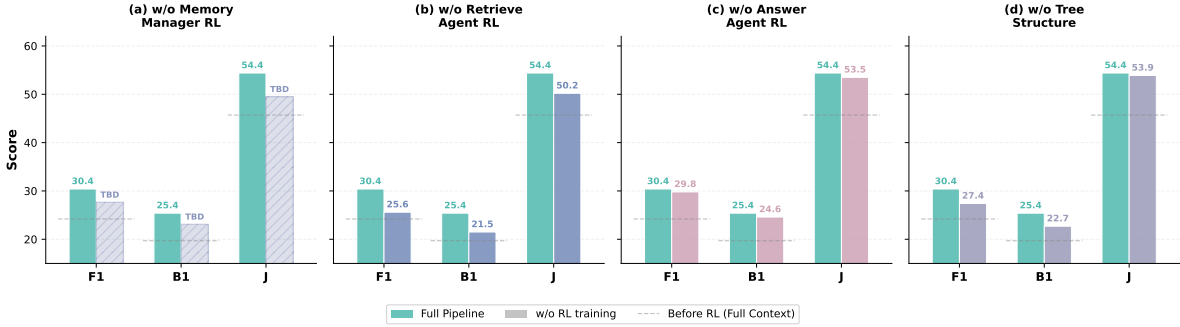


Figure 3: Ablation analysis of STRUCT MEMORY-R1. Each subfigure shows the effect of disabling RL training for one component: (a) Memory Manager, (b) Retrieve Agent, (c) Answer Agent, and (d) replacing tree structure with flat memory. Teal bars show the full pipeline; colored bars show the ablated variant. Grey dashed lines indicate the before-RL baseline (Full Context). Note that “w/o” disables RL fine-tuning for the component, not the component itself. The Memory Manager ablation (hatched bars) is pending completion.

An interesting pattern appears in the retrieval-based baselines. Semantic XPath achieves substantially higher J than both RAG methods on single-hop (45.1 vs. 30.6/32.4) and multi-hop (53.7 vs. 28.1/31.5) despite comparable or lower F1 and B1. This indicates that structured retrieval surfaces contextually richer evidence that helps the model produce semantically correct answers even when lexical overlap with the gold answer is limited. The same F1–J divergence appears in our method relative to RAG: our J scores are consistently higher than RAG across multi-hop, single-hop, and temporal categories, even when lexical metrics are comparable.

We note that our current results use oracle human-judged structured data for memory construction rather than automatically induced structure. This means the Memory Manager component has not yet been trained end-to-end, and the reported numbers may overestimate the performance achievable with fully automatic structure induction. The ablation study below examines the contribution of each component under this setting.

5.2 Ablation Study

Figure 3 shows the effect of disabling RL training for each component. Disabling Retrieve Agent RL (Figure 3b) causes the largest degradation, dropping F1 from 30.4 to 25.6, B1 from 25.4 to 21.5, and J from 54.4 to 50.2. This confirms that learned tree navigation is the most critical component: without it, the system falls back to heuristic retrieval over tree leaves and loses the ability to plan structured search paths.

Disabling Answer Agent RL (Figure 3c) produces only a modest drop (F1 to 29.8, B1 to 24.6, J to 53.5), suggesting that the base Qwen2.5-3B-Instruct model is already a reasonably capable reader when provided with well-retrieved evidence. The primary bottleneck in our pipeline is retrieval quality, not answer generation.

Replacing tree structure with flat memory (Figure 3d) while keeping all RL components yields numbers that closely match Memory R1 (F1 27.4, B1 22.7, J 53.9), as expected since this ablation is architecturally equivalent to Memory R1. The 3.0-point F1 gap between flat and tree-structured retrieval demonstrates the value of hierarchical memory organization as an inductive bias for the retriever.

The Memory Manager RL ablation (Figure 3a) is ongoing and will quantify the contribution of RL-trained memory writing.

6 Discussion

Where tree structure helps. The results confirm that tree-structured memory provides clear advantages for questions requiring multi-step or temporal reasoning, while flat methods retain an edge on open-domain recall. The 9.0-point F1 gain on multi-hop and 13.8-point gain on temporal questions over Memory R1 suggest that the hierarchical grouping of facts by speaker, topic, and session enables retrieval paths that flat scoring cannot replicate. Multi-hop questions benefit because related facts (e.g., a person’s job and the city where they work) are co-located under a shared ancestor, reducing the retriever’s search space. Temporal questions benefit because events are organized chronologically within topic branches, making recency and ordering explicit. At the same time, the dominance of RAG (BM25) on open-domain questions (38.7 F1) highlights that broad keyword coverage remains valuable when questions do not require structural navigation.

Lexical vs. semantic evaluation. The consistent F1–J divergence across baselines is informative. Methods that use structured memory (Semantic XPath, our method) tend to achieve higher LLM-judge accuracy relative to their F1, suggesting that tree-guided retrieval surfaces more contextually complete evidence even when the final answer does not lexically match the gold reference. This pattern reinforces the value of evaluating with both lexical and semantic metrics, and suggests that F1 alone may underestimate the advantage of structured retrieval.

Oracle structure caveat. The current formulation uses oracle structured data for memory construction, meaning the Memory Manager has not yet been trained end-to-end. This represents an important caveat: the reported gains reflect the ceiling of what tree structure can provide given perfect writing, not what an end-to-end system would achieve. Completing the Memory Manager training loop is the most important next step. We expect some degradation with automatically induced structure, but the ablation study shows that the Retrieve Agent RL contributes the largest share of performance—suggesting that even imperfect tree structure, when combined with learned navigation, should outperform flat alternatives.

Modular training. More broadly, the results suggest that structural ambition must be balanced against training stability. The present method uses stable CRUD updates for memory writing, while the Retrieve Agent carries the burden of structure-aware navigation. This separation is cleaner than asking a single policy to both reorganize memory arbitrarily and discover retrieval strategies simultaneously, and is supported by the ablation: disabling Retrieve Agent RL causes the largest performance drop (−4.8 F1), confirming that learned tree navigation is the most critical component.

7 Related Work

Memory-Augmented LLMs and RAG. Retrieval-augmented generation improves factuality and coverage by accessing external knowledge at inference time [4]. Flat nearest-neighbor memory has likewise been used to improve language-model behavior through direct memory lookup [3]. In agent settings, memory is often maintained as a chronological stream of observations, reflections, or summaries [7]. A recent survey [13] provides a comprehensive overview of memory mechanisms for LLM-based agents. These approaches demonstrate the value of persistent memory but treat it as a flat collection, without exploiting the event and topic structure inherent in conversation.

Structured and Hierarchical Memory. Several methods organize memory hierarchically to improve access. RAPTOR [9] uses recursive abstractive processing to build tree-organized retrieval structures. MemWalker [1] navigates memory trees interactively, reading beyond context limits. MemTree [8] explores tree-structured

memory for conversational agents. Lattice [11] introduces LLM-guided hierarchical retrieval over structured memory. Semantic XPath [5] generates path-like queries over memory trees. These works show that hierarchical organization can improve memory access, but typically assume the structure already exists or rely on heuristic construction methods. Our work differs in that the tree is induced incrementally from raw conversation and access over it is optimized as a learned policy using downstream reward.

RL for Search and Memory. Search-R1 treats external search as a policy decision and optimizes reasoning-search trajectories with reinforcement learning [2]. Memory-R1 extends this perspective to persistent memory, training a Memory Manager and an Answer Agent using downstream question-answering reward [12]. These works establish the key principle that information access and memory manipulation should be trained from task outcomes rather than from heuristic rules. However, Search-R1 addresses external search rather than persistent conversational memory, and Memory-R1 operates over flat memory without addressing tree-structured access or updates. Our work inherits this outcome-driven perspective while extending it to tree-structured memory with learned access and update policies.

Gap addressed by this work. Prior work treats memory structure, learned access, and learned updates as separate concerns. Hierarchical memory methods [9, 1, 8, 11] assume or heuristically construct structure but do not learn access policies; RL-based methods [2, 12] learn access policies but operate over flat memory. STRUCT MEMORY-R1 addresses all three within a unified RL framework: it induces tree structure from raw conversation and trains both tree-aware access and update policies from downstream QA reward.

8 Conclusion

We introduced STRUCT MEMORY-R1, a reinforcement-learning framework that learns tree-structured conversational memory from raw interaction history. The framework decomposes the memory-augmented generation problem into three components—a Memory Manager for structural updates, a Retrieve Agent for tree-based search planning, and an Answer Agent for evidence-conditioned response generation—each trained with GRPO from downstream QA reward. On the LoCoMo benchmark, STRUCT MEMORY-R1 achieves the highest overall F1 (30.4) and LLM-judge accuracy (54.4) among non-oracle methods, with +9.0 F1 on multi-hop and +13.8 F1 on temporal questions over the flat-memory Memory-R1 baseline. These gains confirm that hierarchical memory organization is most valuable when questions require navigating relationships among facts across sessions. The framework extends the outcome-driven training principle of Search-R1 and Memory-R1 from flat memory to tree-structured access and updates, providing a principled path toward long-horizon conversational agents with learned structured memory.

9 Limitations

The present formulation of STRUCT MEMORY-R1 has several important limitations. **(1) Oracle structure.** The current results rely on oracle human-judged structure for memory construction. While the Memory Manager is designed for end-to-end training, its RL loop has not yet been completed; the reported gains therefore represent a ceiling on what tree structure can provide. **(2) Fixed schema.** The Memory Manager performs CRUD updates under a stable induced schema but does not yet support general tree restructuring (e.g., splitting or merging branches) during training. **(3) Single-pass retrieval.** The Retrieve Agent emits a single search plan rather than acting interactively one level at a time with intermediate observations. Multi-turn tree traversal could improve recall on complex queries. **(4) No hybrid retrieval.** The method emphasizes explicit tree search rather than dense vector retrieval, so it does not yet address the hybrid regime in which semantic similarity and structural routing are both learned. **(5) Single benchmark.** Evaluation is limited to LoCoMo; broader validation on additional long-horizon conversational benchmarks is needed.

References

- [1] Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. Walking down the memory maze: Beyond context limit through interactive reading. *arXiv preprint arXiv:2310.05029*, 2023.
- [2] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

- [3] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020.
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] Yifan Simon Liu, Ruifan Wu, Liam Gallagher, Jiazhou Liang, Armin Toroghi, and Scott Sanner. Semantic XPath: Structured agentic memory access for conversational AI. *Under Review*, 2026.
- [6] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.
- [7] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- [8] Amin Rezazadeh and Scott Sanner. MemTree: Tree-structured memory for long-horizon conversational agents. *arXiv preprint*, 2024.
- [9] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [11] Ruifan Wu, Yifan Liu, Liam Gallagher, and Scott Sanner. Lattice: LLM-guided hierarchical retrieval for conversational memory. *Under Review*, 2025.
- [12] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Kristian Kersting, Jeff Z Pan, Hinrich Schütze, et al. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.
- [13] Zeyu Zhang, Xiaohe Bo Zhang, Chen Jiang, Bin Liu, Jing Fan, Zhebin Guo, Haihong Mei, Junfeng Fan, Weining Lu, et al. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.